

DM-NAV: Deployable Meta Multi-Agent Reinforcement Learning for Dual-Arm Dynamic Obstacle Avoidance

Anonymous submission

Abstract

Human–Robot Interaction (HRI) is evolving from single-robot assistance toward multi-robot collaborative systems. Multi-Agent Reinforcement Learning (MARL) provides a promising framework for coordinated decision-making in such systems. However, existing MARL-based navigation methods often exhibit limited adaptability when operating around dynamic and unpredictable obstacles, which may result in unsafe or inefficient robot behavior. To address this limitation, we propose Dynamic Meta Navigation (DM-NAV), a Meta-MARL framework for dual-arm robot navigation and collision avoidance in dynamic environments. The DM-NAV algorithm learns a transferable meta policy initialization and adapts it to a specified obstacle-motion task while coordinating the two manipulators. In two simulation-level comparisons with six MARL baselines, the adapted conservative policy completes the 20,000-step horizon and records collision-free execution under both nominal and high-speed sinusoidal motion, which outperforms other models. We further deploy the frozen policy on a physical dual-UR5 system through RGB-D perception and joint-velocity control. 14 physical speed conditions record a soft success rate of 99.4% and a Soft Collision Rate of 0.0% after 10 adaptation iterations.

Introduction and Related Work

Human–Robot Interaction (HRI) is increasingly evolving from single-robot assistance toward collaborative robotic systems in which multiple robots operate in shared and dynamic workspaces. Safe motion generation is fundamental in these settings. Robots must avoid external obstacles, coordinate with one another, and satisfy kinematic and operational constraints at the same time. Classical approaches such as reciprocal velocity obstacles provide efficient collision avoidance under explicit geometric and motion assumptions (van den Berg et al. 2011). Real obstacles, however, can change speed or temporal pattern, making accurate behavior modeling and online trajectory prediction difficult. (Figure 1 presents the physical dual-arm systems that motivate the considered dynamic-obstacle-avoidance setting.)

Deep Reinforcement Learning (DRL) provides an alternative to manually designed interaction rules by learning navigation behavior from experience. Early studies produced socially compliant motion among pedestrians (Chen et al. 2017) and planned among heterogeneous dynamic agents without assuming a fixed behavior model (Everett, Chen, and How

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

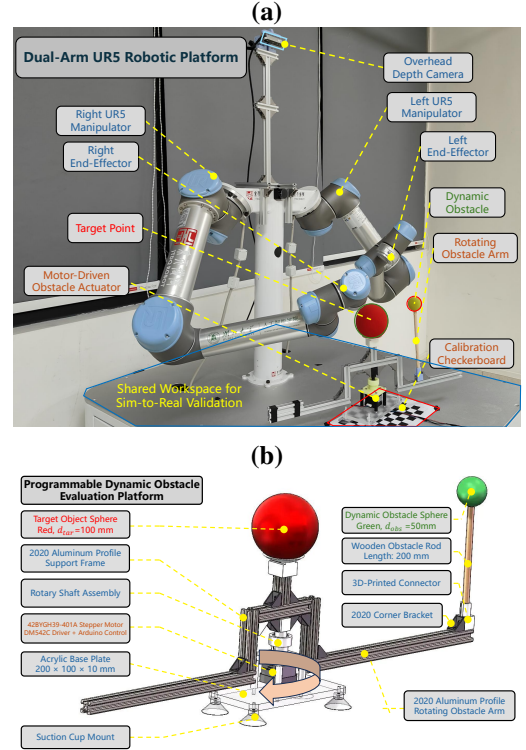


Figure 1: Physical system and dynamic-obstacle mechanism. **(a) Upper:** the dual-UR5 platform, shared workspace, fixed HP60C RGB-D camera, red target, and motor-driven obstacle. **(b) Lower:** the programmable obstacle platform, where a stepper motor rotates the green obstacle sphere on a 0.16-m-radius circular path around the fixed target. The obstacle sphere is 0.05 m in diameter and is supported by a 0.20 m rod; the target sphere is 0.10 m in diameter.

2018). Learning-based methods were subsequently extended to decentralized multi-robot collision avoidance (Long et al. 2018), distributed navigation in crowded scenarios (Fan et al. 2020), and cooperative multi-robot navigation in dynamic environments (Han, Chen, and Hao 2020). Most of these studies address mobile agents. Dual manipulators introduce additional coupling because all links move inside a smaller

shared workspace.

Multi-agent actor-critic methods combine agent-specific actions with information about the joint system. MADDPG uses a joint critic to stabilize learning with multiple actors (Lowe et al. 2017). Evaluations of carefully configured on-policy methods such as MAPPO report strong performance across cooperative benchmarks (Yu et al. 2022). Trust-region formulations study sequential updates for heterogeneous agents (Kuba et al. 2022), while Multi-Agent Transformer expresses cooperative decision-making as sequence modeling and examines transfer to unseen tasks (Wen et al. 2022). These methods primarily address coordination. They do not directly optimize rapid specialization for safety-critical dynamic obstacle motion.

Meta-learning addresses task-specific specialization by optimizing how a policy changes with new data. Model-Agnostic Meta-Learning learns parameters that can be updated through a small number of gradient steps (Finn, Abbeel, and Levine 2017). Gradient-based meta-learning has also enabled continuous adaption in non-stationary and competitive multi-agent environments (Al-Shedivat et al. 2018). For physical robots, meta-reinforcement learning has adapted policies to unseen dynamics, terrain conditions, and hardware variation (Nagabandi et al. 2019). These studies establish the ingredients for rapid adaption, while the present work examines their integration with coordinated dual-arm obstacle avoidance and a physical control interface.

To address the limitation of previous works for dynamic environment adaption, we propose **DM-NAV**, a meta-MARL framework for coordinated dual-arm navigation and dynamic obstacle avoidance. Rather than learning one fixed controller, DM-NAV learns an initialization from a distribution of tasks characterized by different obstacle angular speeds and orbit radii. A joint-state value model and arm-specific stochastic policies are optimized with second-order support and query updates. The initialization is then adapted to a specified target task through a limited interaction budget. The adapted policy is subsequently deployed in a physical closed-loop dual-UR5 system. RGB-D perception and robot-state feedback construct observations containing robot, target, obstacle, and relative-state information. The policy generates coordinated arm commands, while command shaping, runtime guards, and a model-based collision shadow constrain execution. The resulting pipeline connects cross-task learning in MuJoCo, target-task adaption, and physical joint-velocity control. The policy is frozen before deployment, so the physical experiments evaluate transfer rather than online real-world learning.

Our main contributions are summarized as follows:

- We propose a meta-reinforcement-learning method for dual-arm dynamic obstacle avoidance that adapts a learned initialization to previously unseen obstacle-motion tasks with a limited interaction budget.
- We develop an end-to-end dual-UR5 pipeline that integrates cross-task learning in MuJoCo, validation-based target-task adaption, RGB-D perception, and physical closed-loop execution.
- We evaluate conservative and aggressive reward profiles

under four target-task motion laws, compare the adapted method with six MARL baselines, and report quantitative and qualitative physical deployment results.

Method

Overview and Cooperative Task Formulation

DM-NAV is organized into the three stages shown in Figure 2: cross-task meta-policy learning in simulation, gradient-based adaption to a specified obstacle task, and closed-loop execution on the dual-arm platform. Because the embodiment is fixed to two UR5 manipulators, we formulate each task as a cooperative Markov decision process rather than making an equilibrium claim:

$$\mathcal{M}_\tau = (\mathcal{N}, \mathcal{S}, \mathcal{A}, \mathcal{P}_\tau, r_\tau, \gamma, \rho_\tau), \quad \mathcal{N} = \{R, L\}. \quad (1)$$

Here R and L denote the right and left arms, $\mathcal{P}_\tau$ is the task-dependent transition model, r_τ is the shared reward, and ρ_τ is the initial-state distribution. The objective is

$$J_\tau(\theta) = \mathbb{E}_{\pi_\theta, \mathcal{P}_\tau, \rho_\tau} \left[\sum_{t=0}^{T-1} \gamma^t r_\tau(\mathbf{s}_t, \mathbf{a}_t) \right]. \quad (2)$$

Meta-Training for DM-NAV

The left stage of Figure 2 learns an initialization that can be specialized with limited target-task data. At meta-iteration m , a batch of tasks is sampled from $p_{\text{train}}(\tau)$. For task τ_i , a support rollout D_i^S initializes K_m differentiable inner updates:

$$\begin{aligned} \phi_i^{(0)} &= \theta, \\ \mathbf{g}_i^{(k)} &= \nabla_{\phi_i^{(k)}} \mathcal{L}_{\text{PPO}}^S(\phi_i^{(k)}; D_i^S), \\ \phi_i^{(k+1)} &= \phi_i^{(k)} - \alpha \mathbf{g}_i^{(k)}, \quad k = 0, \dots, K_m - 1. \end{aligned} \quad (3)$$

A separate query rollout D_i^Q , collected with the adapted fast weights $\phi_i^{(K_m)}$, defines

$$\begin{aligned} \mathcal{L}_{\text{meta}}(\theta) &= \frac{1}{B} \sum_{i=1}^B \mathcal{L}_{\text{PPO}}^Q(\phi_i^{(K_m)}; D_i^Q), \\ \theta &\leftarrow \theta - \beta \nabla_\theta \mathcal{L}_{\text{meta}}. \end{aligned} \quad (4)$$

The inner-gradient graph is retained when differentiating the query objective, so the outer update includes the dependency of $\phi_i^{(K_m)}$ on θ . Running observation statistics are estimated from training rollouts and stored with the learned initialization θ^* . The number of inner updates increases from one to three during training; the exact schedule appears in the Experimental Setup. Algorithm 1 summarizes task sampling, support adaption, query evaluation, and the second-order outer update.

Target-Task adaption for DM-NAV

The middle stage of Figure 2 specializes the learned initialization before evaluation or physical transfer. For a desired task τ_{tar} , observation statistics are frozen and adaption starts from a selected meta-training checkpoint θ_{base} . At iteration

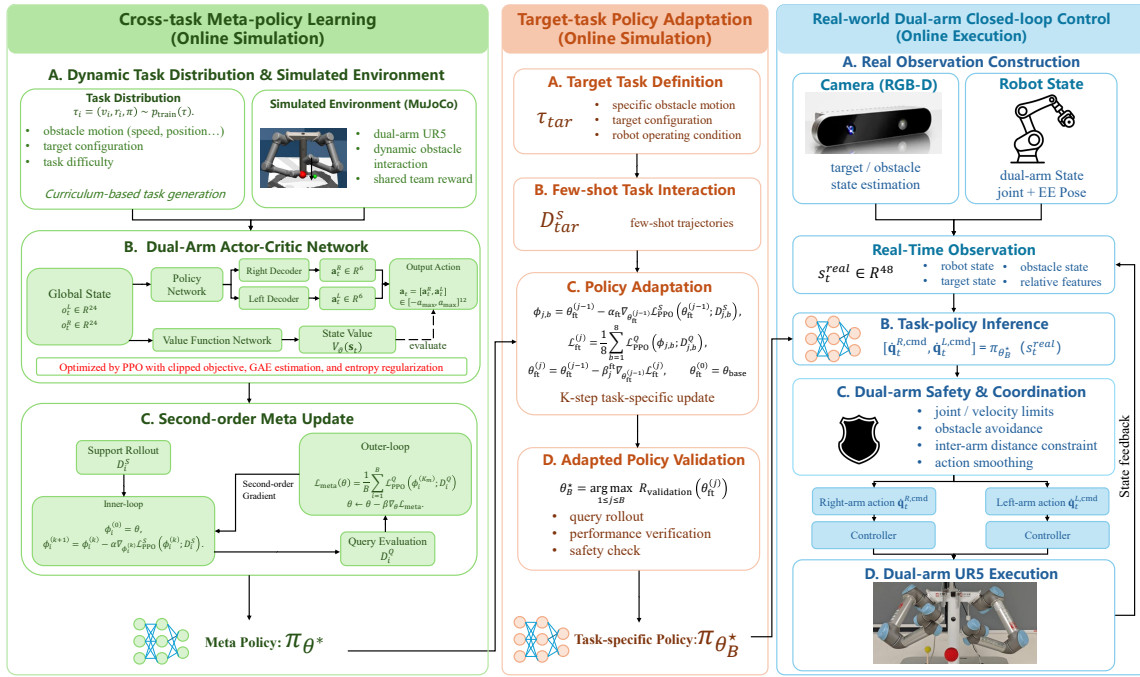


Figure 2: DM-NAV from simulation to physical execution. **Left:** meta-training samples obstacle-motion tasks and forms separate support and query trajectories to learn a transferable initialization. **Middle:** target-task adaption updates that initialization for a specified motion law and selects a checkpoint with held-out validation rollouts. **Right:** RGB-D detections and robot feedback form the policy observation; two arm-specific actors generate commands that pass through command shaping and runtime safety checks before transmission to the UR5 controllers.

j , eight task instances provide independent support and query trajectories. One inner update is computed for each instance:

$$\begin{aligned} \phi_{j,b} &= \theta_{\text{ft}}^{(j-1)} - \alpha_{\text{ft}} \nabla_{\theta} \mathcal{L}_{\text{PPO}}^S(\theta_{\text{ft}}^{(j-1)}; D_{j,b}^S), \\ \mathcal{L}_{\text{ft}}^{(j)} &= \frac{1}{8} \sum_{b=1}^8 \mathcal{L}_{\text{PPO}}^Q(\phi_{j,b}; D_{j,b}^Q), \\ \theta_{\text{ft}}^{(j)} &= \theta_{\text{ft}}^{(j-1)} - \beta_{\text{ft}} \nabla_{\theta} \mathcal{L}_{\text{ft}}^{(j)}, \quad \theta_{\text{ft}}^{(0)} = \theta_{\text{base}}. \end{aligned} \quad (5)$$

For a maximum budget $B \in \{1, 5, 10\}$, fixed validation seeds select the strongest checkpoint available within that budget:

$$\theta_B^* = \arg \max_{1 \leq j \leq B} R_{\text{validation}}(\theta_{\text{ft}}^{(j)}). \quad (6)$$

The zero-iteration condition uses θ_{base} . Evaluation seeds never participate in checkpoint selection, and no test trajectory is reused as adaption data. Algorithm 2 connects the adaption budget, validation-based checkpoint selection, deterministic testing, and metric aggregation. Both reward profiles share a 48-dimensional global observation, a 24-dimensional local observation and a six-dimensional action for each arm. They also share the PPO coefficients and target-task adaption interface. Appendix Table 1 consolidates these environment, network, and adaption parameters; Appendix Table 2 separately compares the optimization and interaction budgets. Algorithms 1 and 2 summarize the two computational phases. The first constructs the reusable initialization from task-wise support and query rollouts. The second

starts from that initialization, performs target-task updates, validates intermediate checkpoints, and evaluates only the selected model.

Physical Deployment Setup and Direct Transfer

The deployment system comprises two UR5 manipulators mounted on a shared rigid base and an ASCAMERA HP60C RGB-D camera fixed above the common workspace in an eye-to-hand configuration. The red target sphere has a diameter of 0.10 m. The moving green obstacle sphere has a diameter of 0.05 m and is mounted on a 0.20 m wooden rod. A 42BYGH39-401A stepper motor, DM542C driver, and Arduino controller rotate the obstacle about the target axis on a circular orbit of radius 0.16 m. Figure 1 shows the shared workspace and the programmable obstacle mechanism. The geometry remains fixed across the physical speed sweep, while the motor command generates the constant- and sinusoidal-velocity conditions. To align camera observations with the robot workspace, the fixed world-to-camera transform is estimated with a 3×4 ArUco GridBoard attached to the left end effector. Calibration uses a 25-marker dictionary with 5×5 binary patterns, 0.06 m marker sides, and 0.006 m marker spacing. Fifteen retained poses give recorded translation and rotation residuals of approximately 0.016 m and 0.015 rad, respectively. The estimated world-to-camera translation is approximately (0.0603, -0.0082, 1.7294) m. The transform is checked in RViz and through the ROS 2 transform tree before physical testing.

Algorithm 1: DM-NAV Meta-Training

Require: Task distribution $p_{\text{train}}(\tau)$; iterations M ; batch size B
Require: Support/query lengths H_S, H_Q ; inner steps K_m ; rates α, β
Ensure: Meta-initialization θ^* and observation statistics $\mathcal{Z}$

- 1: Initialize dual-actor/value parameters θ , optimizer, and $\mathcal{Z}$
- 2: **for** $m = 1$ to M **do**
- 3: Schedule the outer rate β_m , entropy weight η_m , and inner count K_m
- 4: Sample $\tau_i = (v_i, r_i, \pi) \sim p_{\text{train}}(\tau)$ for $i = 1, \dots, B$
- 5: **for** each task τ_i **do**
- 6: Collect support rollout D_i^S and initialize $\phi_i^{(0)} \leftarrow \theta$
- 7: **for** $k = 0$ to $K_m - 1$ **do**
- 8: $\phi_i^{(k+1)} \leftarrow \phi_i^{(k)} - \alpha \nabla_{\phi_i^{(k)}} \mathcal{L}_{\text{PPO}}^S(\phi_i^{(k)}; D_i^S, \eta_m)$
- 9: **end for**
- 10: Collect an independent query rollout D_i^Q with $\pi_{\phi_i^{(K_m)}}$
- 11: $\ell_i^Q \leftarrow \mathcal{L}_{\text{PPO}}^Q(\phi_i^{(K_m)}; D_i^Q, \eta_m)$
- 12: **end for**
- 13: $\mathcal{L}_{\text{meta}} \leftarrow B^{-1} \sum_i \ell_i^Q$
- 14: Update θ through the retained inner-gradient graphs
- 15: **end for**
- 16: **return** $\theta^* \leftarrow \theta$

RGB and depth frames are approximately time synchronized. YOLO first localizes the red target and green obstacle in the RGB image. Within each detection box, an HSV mask and largest-connected-component operation reject background pixels; depth gating retains plausible samples, and their median provides the axial distance. With pixel center (u, v) and camera intrinsics (f_x, f_y, c_x, c_y) , the camera-frame point and its world-frame counterpart are computed as

$$\begin{aligned} z_c &= \text{median}_{(u', v') \in \Omega} D(u', v'), \\ \mathbf{p}_c &= [(u - c_x)z_c/f_x \quad (v - c_y)z_c/f_y \quad z_c]^\top, \\ \mathbf{p}_w &= R_{wc}\mathbf{p}_c + \mathbf{t}_{wc}. \end{aligned} \quad (7)$$

Because the median depth lies near the visible sphere surface, the implementation applies an axial center correction using the known sphere radii, 0.05 m for the red target and 0.025 m for the green obstacle. This correction assumes that the detected component belongs to the visible sphere surface and approximates the center along the camera ray. The world-frame horizontal coordinates are then filtered with exponential smoothing and jump protection, while the height coordinate uses a window median, deadband, and persistent-change check. The resulting target and obstacle states are published through ROS 2 and TF, combined with UR5 feedback, and arranged into the 48-dimensional policy observation defined in the supplementary material.

For the reported deployment, validation selects θ_{10}^* from the first ten fine-tuning iterations. The selected policy is then frozen; no gradient update or fine-tuning is performed on physical data. The MoveIt and ROS 2 control stack is used to commission the manipulators and verify the calibrated workspace. During the reported policy runs, the on-line controller uses a joint-velocity interface. The two six-dimensional policy outputs are converted to right- and left-arm commands $\dot{\mathbf{q}}^R$ and $\dot{\mathbf{q}}^L$.

Algorithm 2: DM-NAV Target-Task adaption

Require: $\theta_{\text{base}}, \mathcal{Z}$; target task τ_{tar} ; budgets $\mathcal{K} = \{0, 1, 5, 10\}$
Require: Validation seeds $\{100, 101\}$; test seeds $\{0, 1, 2, 3, 4\}$
Ensure: Selected policies $\{\theta_B^*\}$ and metrics $\{\mathcal{M}^{(B)}\}$

- 1: Freeze $\mathcal{Z}$ and set $\theta_{\text{ft}}^{(0)} \leftarrow \theta_{\text{base}}$
- 2: **for** $j = 1$ to 10 **do**
- 3: Collect eight independent support/query pairs for τ_{tar}
- 4: Compute one inner update per pair and update $\theta_{\text{ft}}^{(j)}$ using (5)
- 5: Evaluate checkpoint j with validation seeds 100 and 101
- 6: **end for**
- 7: **for** each budget $B \in \mathcal{K}$ **do**
- 8: Set $\theta_0^* \leftarrow \theta_{\text{base}}$ if $B = 0$; otherwise apply (6)
- 9: **for** each deterministic test seed $e \in \{0, 1, 2, 3, 4\}$ **do**
- 10: Reset τ_{tar} ; initialize the two previous commands
- 11: **while** the episode remains active **do**
- 12: Normalize $\mathbf{s}_t$; decode and filter both arm commands
- 13: Step the environment and record task and safety metrics
- 14: **end while**
- 15: **end for**
- 16: **end for**
- 17: **return** $\{\theta_B^*\}_{B \in \mathcal{K}}$

Decision timing and velocity-command shaping. Both physical launch packages set the upper-level policy period to $\Delta t_\pi = 0.02$ s, or 50 Hz. This setting specifies the learned-policy decision rate. The internal UR servo rate remains determined by the ROS controller manager and the UR control cabinet. A separate command watchdog republishes the latest valid velocity every $\Delta t_w = 0.01$ s, corresponding to 100 Hz. If no new valid command arrives for 0.10 s, it continuously publishes zero velocity at the watchdog rate.

Let $\mathbf{a}_t$ denote either arm’s six-dimensional network output and $\dot{\mathbf{q}}_t^{\text{cmd}}$ the resulting command. The physical command path clips the action to $[-1, 1]$, applies the policy/command low-pass filters, removes magnitudes below the 0.001 rad/s deadband, and enforces velocity, acceleration, and jerk limits:

$$\begin{aligned} \bar{\mathbf{a}}_t &= \text{clip}(\mathbf{a}_t, -1, 1), \\ \|\dot{\mathbf{q}}_t^{\text{cmd}}\|_\infty &\leq \dot{q}_{\text{max}}, \\ \frac{\|\Delta \dot{\mathbf{q}}_t^{\text{cmd}}\|_\infty}{\Delta t_\pi} &\leq \ddot{q}_{\text{max}}, \quad \frac{\|\Delta^2 \dot{\mathbf{q}}_t^{\text{cmd}}\|_\infty}{\Delta t_\pi^2} \leq j_{\text{max}}. \end{aligned} \quad (8)$$

Here $\dot{q}_{\text{max}}$, $\ddot{q}_{\text{max}}$, and j_{max} denote the configured velocity, acceleration, and jerk limits. The physical scripts use $\dot{q}_{\text{max}} = 0.08$ rad/s by default, although the deployment configuration can override this value. At startup, both packages hold zero velocity for 2 s and then progressively blend in the policy command.

Runtime guards and collision-triggered stopping. Before forwarding a command, the runtime verifies the freshness and numerical validity of the ROS 2 joint-state stream and monitors tracking error, measured joint speed, and UR speed scaling. Stale or invalid feedback, an exceeded bound, or a rejected six-dimensional watchdog command produces zero velocity. Collision-triggered stopping is mandatory at launch. In parallel, the MuJoCo shadow checks obstacle and support contact, end-effector and elbow proximity, ground

(a) Conservative policy					(b) Aggressive policy				
Motion	Metric	0 updates	5 updates	10 updates	Motion	Metric	0 updates	5 updates	10 updates
Slow	$SR \uparrow$	0.318 ± 0.048	0.438 ± 0.013	0.471 ± 0.008	Slow	$SR \uparrow$	0.455 ± 0.057	0.625 ± 0.015	0.693 ± 0.009
	$SSR \uparrow$	0.805 ± 0.032	0.952 ± 0.007	0.994 ± 0.002		$SSR \uparrow$	0.850 ± 0.039	0.966 ± 0.007	0.994 ± 0.002
	$SCR \downarrow$	0.018 ± 0.012	0.006 ± 0.003	0.000 ± 0.000		$SCR \downarrow$	0.043 ± 0.017	0.016 ± 0.005	0.000 ± 0.000
	$d_{tar} \downarrow$ (m)	0.169 ± 0.017	0.128 ± 0.005	0.116 ± 0.003		$d_{tar} \downarrow$ (m)	0.150 ± 0.020	0.116 ± 0.005	0.106 ± 0.003
	$d_{obs}^* \text{ (m)}$	0.232 ± 0.013	0.249 ± 0.004	0.257 ± 0.003		$d_{obs}^* \text{ (m)}$	0.210 ± 0.016	0.236 ± 0.005	0.246 ± 0.003
Fast	$SR \uparrow$	0.238 ± 0.066	0.371 ± 0.015	0.432 ± 0.009	Fast	$SR \uparrow$	0.372 ± 0.079	0.558 ± 0.018	0.654 ± 0.011
	$SSR \uparrow$	0.704 ± 0.052	0.892 ± 0.011	0.994 ± 0.004		$SSR \uparrow$	0.758 ± 0.061	0.913 ± 0.011	0.994 ± 0.004
	$SCR \downarrow$	0.052 ± 0.025	0.024 ± 0.006	0.000 ± 0.000		$SCR \downarrow$	0.084 ± 0.029	0.041 ± 0.009	0.000 ± 0.000
	$d_{tar} \downarrow$ (m)	0.197 ± 0.027	0.147 ± 0.007	0.117 ± 0.004		$d_{tar} \downarrow$ (m)	0.178 ± 0.030	0.132 ± 0.007	0.108 ± 0.004
	$d_{obs}^* \text{ (m)}$	0.219 ± 0.023	0.241 ± 0.005	0.257 ± 0.004		$d_{obs}^* \text{ (m)}$	0.197 ± 0.026	0.228 ± 0.006	0.246 ± 0.004
Sinusoidal	$SR \uparrow$	0.298 ± 0.053	0.426 ± 0.012	0.474 ± 0.007	Sinusoidal	$SR \uparrow$	0.433 ± 0.063	0.612 ± 0.016	0.691 ± 0.009
	$SSR \uparrow$	0.778 ± 0.038	0.938 ± 0.008	0.994 ± 0.003		$SSR \uparrow$	0.826 ± 0.045	0.954 ± 0.008	0.994 ± 0.003
	$SCR \downarrow$	0.026 ± 0.016	0.010 ± 0.004	0.000 ± 0.000		$SCR \downarrow$	0.055 ± 0.021	0.022 ± 0.006	0.000 ± 0.000
	$d_{tar} \downarrow$ (m)	0.176 ± 0.020	0.132 ± 0.005	0.115 ± 0.003		$d_{tar} \downarrow$ (m)	0.157 ± 0.023	0.119 ± 0.005	0.105 ± 0.003
	$d_{obs}^* \text{ (m)}$	0.228 ± 0.016	0.247 ± 0.004	0.257 ± 0.003		$d_{obs}^* \text{ (m)}$	0.206 ± 0.019	0.234 ± 0.005	0.247 ± 0.003
Random	$SR \uparrow$	0.276 ± 0.058	0.408 ± 0.014	0.480 ± 0.006	Random	$SR \uparrow$	0.412 ± 0.070	0.592 ± 0.017	0.697 ± 0.010
	$SSR \uparrow$	0.748 ± 0.044	0.923 ± 0.009	0.994 ± 0.003		$SSR \uparrow$	0.802 ± 0.052	0.941 ± 0.009	0.994 ± 0.003
	$SCR \downarrow$	0.036 ± 0.020	0.015 ± 0.005	0.000 ± 0.000		$SCR \downarrow$	0.067 ± 0.025	0.030 ± 0.007	0.000 ± 0.000
	$d_{tar} \downarrow$ (m)	0.185 ± 0.023	0.138 ± 0.006	0.115 ± 0.004		$d_{tar} \downarrow$ (m)	0.166 ± 0.026	0.123 ± 0.006	0.106 ± 0.004
	$d_{obs}^* \text{ (m)}$	0.224 ± 0.019	0.244 ± 0.005	0.257 ± 0.004		$d_{obs}^* \text{ (m)}$	0.202 ± 0.022	0.231 ± 0.006	0.247 ± 0.004

Table 1: Target-task adaption under the Slow, Fast, Sinusoidal, and Random motion protocols defined in the Target-Task Motion Protocols subsection and summarized in Appendix Table 3. Panels (a) and (b) compare conservative and aggressive policies horizontally; columns report 0, 5, and 10 fine-tuning iterations. Each cell is the mean $\pm$ population standard deviation over deterministic seeds 0 through 4. Fine-tuning uses budgets 5 and 10 select the best fixed-validation checkpoint available within the corresponding budget. $\uparrow$ and $\downarrow$ indicate that larger and smaller values are preferred, respectively. The starred d_{obs}^* metric is an operating-point descriptor rather than a quantity with a universal monotonic preference.

contact, inter-arm body and tip distance, and simulated contacts. The second runtime package resets this shadow from measured joints at every 50-Hz policy tick and predicts 0.02 s ahead; predicted contact sends zero joint velocity, stops the obstacle motor, and enters an error state.

Safety boundary. The current joint-velocity branch does not perform joint-specific predictive braking against the true lower and upper limit of every UR5 joint before each command. Joint bounds remain present in the MuJoCo model and UR controller, while the separate ROS trajectory branch uses a default software bound of $\pm 2\pi$. The online velocity branch primarily relies on the shadow model, command-rate limits, and UR cabinet protections. In model-based scene mode, obstacle checking follows the prescribed trajectory rather than an independent measurement of the physical obstacle. A deviation of the real mechanism from that trajectory is therefore outside the shadow model’s observation. Finally, the software stop publishes zero velocity and stops the obstacle motor; it does not remove power from the manipulators. Physical emergency stopping remains the responsibility of the teach pendant and hardware safety circuit.

Experiments

Simulation and Control Setup

Experiments use MuJoCo 3.1.6 and a rodless dual-UR5 model. The simulator integrates the dynamics with RK4 at a physical step of 0.002 s. Two physics steps are executed for each policy action, which gives a control period of $\Delta t = 0.004$ s and an effective policy rate of 250 Hz. These settings are shared by both reward profiles. Training episodes contain at most 5,000 control steps, or approximately 20 s. Formal tests extend the horizon to 20,000 steps, corresponding to about 80 s. A collision ends the current trajectory early. Target success is logged without terminating the formal test. Execution continues until collision or the maximum horizon. Details about evaluation metrics and baselines are available in supplemental materials

Results for MuJoCo Simulation

adaption Across Obstacle Motions Table 1 compares the conservative and aggressive reward profiles before adaption and after five or ten target-task updates. Averaged across the four motion tasks, conservative SR increases from 28.3% to 41.1% and 46.4%, while aggressive SR rises from 41.8% to 59.7% and 68.4%. Approximately 71% and 67% of the respective total gains occur within the first five updates, indicating that adaption is most effective at an early stage.

Metric	DDPG	DADDPG	DQN	TDQN	AC	PPO	DM-NAV(Ours)
<i>Nominal sinusoidal motion, 0.20 to 0.40 rad/s</i>							
Survival rate (%)	66.203 ± 28.407	77.412 ± 21.134	49.324 ± 27.562	58.684 ± 24.826	37.108 ± 15.925	92.134 ± 13.818	100.000 ± 0.000
$SR \uparrow$	0.085 ± 0.052	0.142 ± 0.071	0.038 ± 0.029	0.067 ± 0.041	0.021 ± 0.018	0.268 ± 0.096	0.783 ± 0.006
$SSR \uparrow$	0.410 ± 0.180	0.526 ± 0.144	0.461 ± 0.190	0.498 ± 0.161	0.273 ± 0.122	0.721 ± 0.115	0.994 ± 0.000
$SCR \downarrow$	0.380 ± 0.150	0.290 ± 0.120	0.520 ± 0.170	0.440 ± 0.160	0.610 ± 0.140	0.150 ± 0.080	0.000 ± 0.000
$d_{tar} \downarrow$ (m)	0.310 ± 0.090	0.265 ± 0.073	0.338 ± 0.082	0.306 ± 0.076	0.374 ± 0.064	0.198 ± 0.052	0.087 ± 0.000
d_{obs}^* (m)	0.292 ± 0.055	0.318 ± 0.049	0.271 ± 0.045	0.286 ± 0.047	0.254 ± 0.035	0.341 ± 0.042	0.235 ± 0.000
<i>High-speed sinusoidal motion, 0.68 to 0.88 rad/s</i>							
Survival rate (%)	54.171 ± 29.534	64.814 ± 25.542	37.733 ± 21.866	45.936 ± 22.643	28.074 ± 13.707	84.362 ± 17.131	100.000 ± 0.000
$SR \uparrow$	0.041 ± 0.031	0.073 ± 0.044	0.015 ± 0.014	0.028 ± 0.022	0.008 ± 0.009	0.156 ± 0.074	0.404 ± 0.000
$SSR \uparrow$	0.312 ± 0.149	0.408 ± 0.137	0.342 ± 0.171	0.381 ± 0.155	0.184 ± 0.094	0.604 ± 0.129	0.994 ± 0.000
$SCR \downarrow$	0.510 ± 0.160	0.420 ± 0.150	0.650 ± 0.150	0.570 ± 0.160	0.730 ± 0.120	0.260 ± 0.110	0.000 ± 0.000
$d_{tar} \downarrow$ (m)	0.378 ± 0.102	0.326 ± 0.085	0.405 ± 0.090	0.369 ± 0.083	0.432 ± 0.071	0.244 ± 0.063	0.117 ± 0.000
d_{obs}^* (m)	0.276 ± 0.061	0.294 ± 0.056	0.249 ± 0.047	0.262 ± 0.050	0.231 ± 0.038	0.312 ± 0.047	0.261 ± 0.000

Table 2: Fast adaption (10 steps) results on same task over 10 random seeds. All MARL baselines are pretrained for 4000 epochs while our model is pre-trained with 400 meta-training epochs (batch size is 8).

Improvements are consistent across all motion laws. Fast motion remains the most difficult deterministic case, with ten-update SR values of 43.2% and 65.4%, likely because its maximum speed of 1.28 rad/s substantially exceeds the 0.48 rad/s meta-training limit. Random motion nevertheless achieves the highest ten-update SR in both profiles, suggesting that temporal variability alone does not determine task difficulty. At ten updates, every reported policy-task pair reaches $SSR=99.4\%$ and a displayed SCR of 0%, while SR variability across evaluation seeds is substantially reduced. These results describe rollout-level variability rather than uncertainty across independently trained policies. Moreover, because each budget reports its best validation-selected checkpoint, the monotonic averages demonstrate the benefit of a larger selection budget rather than guaranteed improvement after every gradient update.

Conservative and Aggressive Operating Points The distance rows in Table 1 expose the intended operating difference. After ten updates, conservative d_{tar} lies between 0.115 and 0.117 m, with d_{obs} fixed at 0.257 m to the reported precision. Aggressive d_{tar} falls between 0.105 and 0.108 m, while d_{obs} lies between 0.246 and 0.247 m. Across the four tasks, the aggressive policy is approximately 9.5 mm closer to the target, whereas the conservative policy retains about 10.5 mm more end-effector clearance from the obstacle.

This direction agrees with the reward design. Higher target and success weights encourage closer approach. Earlier safety barriers and stronger motion regularization preserve separation. The comparison remains associative because the two profiles start from checkpoints chosen by different criteria; it should not be interpreted as a reward-only ablation.

Comparison with MARL Baselines with meta learning

Table 2 compares complete learning and control pipelines under nominal and high-speed sinusoidal obstacle motion. In the nominal panel, PPO is the strongest baseline, with a survival rate of $(92.134 \pm 13.818)\%$, $SR= (26.8 \pm 9.6)\%$,

and $SSR= (72.1 \pm 11.5)\%$. The adapted conservative DM-NAV policy reaches $(100.000 \pm 0.000)\%$ survival, $SR= (78.3 \pm 0.6)\%$, and $SSR= (99.4 \pm 0.0)\%$, while its displayed SCR is 0%.

The high-speed panel preserves the same ordering. PPO records $(84.362 \pm 17.131)\%$ survival, $SR= (15.6 \pm 7.4)\%$, and $SSR= (60.4 \pm 12.9)\%$. DM-NAV maintains full-horizon survival and records $SR= 40.4\%$, $SSR= 99.4\%$, and $SCR= 0\%$. The distance metrics identify different approach and clearance operating points, so d_{obs} should not be interpreted independently of success, collision, and survival. Because the training and adaption pipelines are not identical, the table supports a system-level comparison rather than a causal claim about meta-learning alone.

Results for Real-World Implementation

Physical Deployment Results Table 3 reports direct physical execution with the selected policy frozen. In the constant-velocity panel, each column heading is the commanded obstacle angular speed. The sinusoidal headings are condition identifiers inherited from the experimental record. They do not claim that the instantaneous speed remains equal to the displayed value throughout a sinusoidal run. Under constant velocity, SR rises from 38.0% at 0.18 rad/s to 49.1% at 0.68 rad/s, then changes to 47.2% at 0.78 rad/s. The sinusoidal panel reaches its maximum SR of 51.1% at the 0.28 label and its minimum of 24.5% at the 0.18 label. Neither sequence is monotonic, indicating that phase and temporal coordination also affect the recorded approach behavior. All fourteen conditions record $SSR= 99.4\%$ and $SCR= 0.0\%$. Mean target distance remains between 0.114 and 0.119 m, while mean end-effector clearance from the obstacle lies between 0.257 and 0.270 m. Each column summarizes one 20,000-step continuous record, corresponding to approximately 400 s at the 50-Hz policy rate. Adjacent control steps are correlated, so the table does not represent fourteen independent estimates of physical reliability.

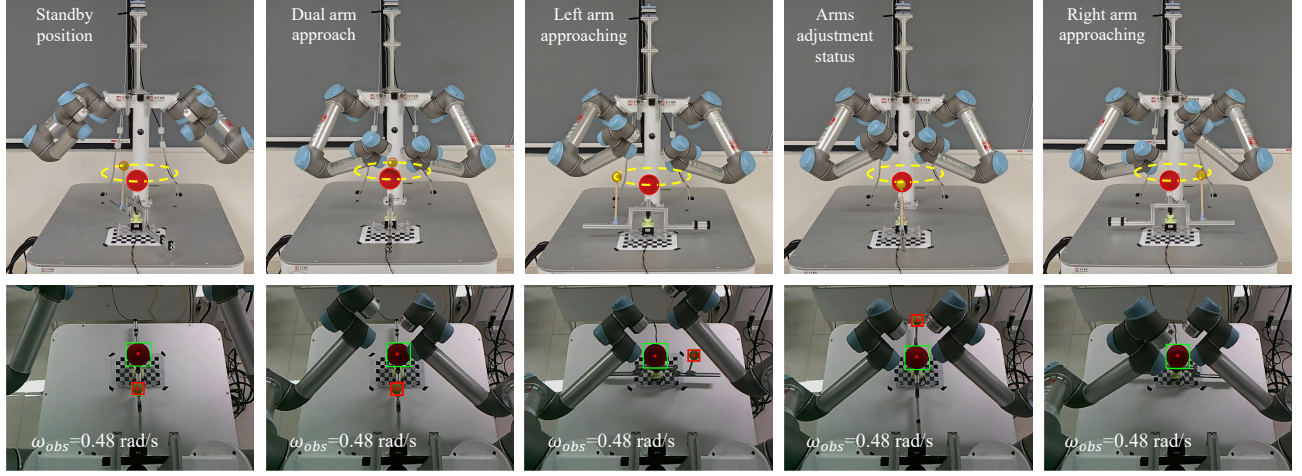


Figure 3: Chronological physical execution of the conservative policy under a constant 0.48,rad/s obstacle command. The composite contains five time-ordered columns. Within each column, the upper row shows the external workspace and the lower row presents the corresponding overhead RGB-D view. Manually assigned stages identify standby, dual-arm entry, left-arm approach, coordinated adjustment, and right-arm approach. The dashed circle marks the 0.16,m obstacle orbit, while colored overlays indicate the detected target and obstacle.

Real-time max speed (rad/s)	0.18	0.28	0.38	0.48	0.58	0.68	0.78
<i>(a) Constant-velocity motion</i>							
$SR \uparrow$	0.380	0.406	0.412	0.461	0.479	0.491	0.472
$SSR \uparrow$	0.994	0.994	0.994	0.994	0.994	0.994	0.994
$SCR \downarrow$	0.000	0.000	0.000	0.000	0.000	0.000	0.000
$d_{tar} \downarrow$ (m)	0.118	0.117	0.116	0.114	0.115	0.115	0.116
d_{obs}^* (m)	0.265	0.262	0.259	0.258	0.258	0.257	0.257
<i>(b) Sinusoidal-velocity motion</i>							
$SR \uparrow$	0.245	0.511	0.440	0.461	0.451	0.482	0.469
$SSR \uparrow$	0.994	0.994	0.994	0.994	0.994	0.994	0.994
$SCR \downarrow$	0.000	0.000	0.000	0.000	0.000	0.000	0.000
$d_{tar} \downarrow$ (m)	0.119	0.117	0.116	0.115	0.115	0.115	0.116
d_{obs}^* (m)	0.270	0.259	0.260	0.258	0.258	0.257	0.257

Table 3: Physical obstacle-speed sweep after 10 target-task adaptations without physical fine-tuning. The upper panel reports commanded constant angular speeds. The lower-panel headings are manually assigned sinusoidal-condition labels from the experimental record rather than constant instantaneous speeds. SR , SSR , and SCR follow ; d_{tar} and d_{obs} follow . Each entry summarizes one continuous 20,000-step record.

Qualitative Physical Execution Figure 3 complements the aggregate metrics with a chronological physical execution sequence. The conservative policy is evaluated under a constant obstacle command of 0.48,rad/s. The upper row shows the external workspace, while the lower row presents the corresponding overhead RGB-D view. The sequence qualitatively demonstrates that perception, policy inference, and velocity control remain integrated throughout the coordinated dual-arm approach. It represents one continuous physical run rather than a set of independent trials.

Conclusion and Future Work

We presented DM-NAV, a meta-reinforcement-learning framework that combines dual-arm policy learning, joint-state value estimation, second-order meta-updates, and co-operative reward design for dynamic obstacle avoidance with a dual-UR5 system. The evaluation fixes the task geometry while varying obstacle motion through four reproducible profiles. Target-task fine-tuning improves the recorded success and soft-collision metrics for both policy configurations. The aggressive policy reaches positions closer to the target, whereas the conservative policy maintains greater end-effector clearance from the moving obstacle. The adapted conservative policy achieves nonzero strict success and full-horizon survival in both sinusoidal baseline settings. The validation-selected policy is further transferred to the physical platform without real-world gradient updates. Across fourteen recorded speed conditions, SSR remains 99.4%, SCR remains 0.0%, d_{tar} ranges from 0.114 to 0.119 m, and d_{obs} ranges from 0.257 to 0.270 m. These results demonstrate stable simulation-to-real execution under the evaluated dynamic-obstacle conditions. The current results primarily provide system-level evidence based on fixed training and checkpoint-selection settings.

Future work will repeat complete training with multiple independent seeds, unify checkpoint-selection rules, more closely match baseline interfaces and computational budgets, and conduct repeated physical trials with confidence-interval reporting. Extending the deployment layer with perception-aware obstacle uncertainty, joint-limit prediction, physical obstacle sensing, and hardware emergency-stop mechanisms will further strengthen real-world safety and generalization.

References

- Al-Shedivat, M.; Bansal, T.; Burda, Y.; Sutskever, I.; Mordatch, I.; and Abbeel, P. 2018. Continuous Adaptation via Meta-Learning in Nonstationary and Competitive Environments. In *International Conference on Learning Representations*.
- Chen, Y. F.; Everett, M.; Liu, M.; and How, J. P. 2017. Socially Aware Motion Planning with Deep Reinforcement Learning. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 1343–1350.
- Everett, M.; Chen, Y. F.; and How, J. P. 2018. Motion Planning Among Dynamic, Decision-Making Agents with Deep Reinforcement Learning. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 3052–3059.
- Fan, T.; Long, P.; Liu, W.; and Pan, J. 2020. Distributed Multi-Robot Collision Avoidance via Deep Reinforcement Learning for Navigation in Complex Scenarios. *The International Journal of Robotics Research*, 39(7): 856–892.
- Finn, C.; Abbeel, P.; and Levine, S. 2017. Model-Agnostic Meta-Learning for Fast Adaptation of Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, 1126–1135. PMLR.
- Han, R.; Chen, S.; and Hao, Q. 2020. Cooperative Multi-Robot Navigation in Dynamic Environment with Deep Reinforcement Learning. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 448–454.
- Kuba, J. G.; Chen, R.; Wen, M.; Wen, Y.; Sun, F.; Wang, J.; and Yang, Y. 2022. Trust Region Policy Optimisation in Multi-Agent Reinforcement Learning. In *International Conference on Learning Representations*.
- Long, P.; Fan, T.; Liao, X.; Liu, W.; Zhang, H.; and Pan, J. 2018. Towards Optimally Decentralized Multi-Robot Collision Avoidance via Deep Reinforcement Learning. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 6252–6259.
- Lowe, R.; Wu, Y.; Tamar, A.; Harb, J.; Abbeel, P.; and Mordatch, I. 2017. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In *Advances in Neural Information Processing Systems*, volume 30, 6379–6390. Curran Associates, Inc.
- Nagabandi, A.; Clavera, I.; Liu, S.; Fearing, R. S.; Abbeel, P.; Levine, S.; and Finn, C. 2019. Learning to Adapt in Dynamic, Real-World Environments through Meta-Reinforcement Learning. In *International Conference on Learning Representations*.
- van den Berg, J.; Guy, S. J.; Lin, M. C.; and Manocha, D. 2011. Reciprocal n -Body Collision Avoidance. In *Robotics Research: The 14th International Symposium ISRR*, volume 70 of *Springer Tracts in Advanced Robotics*, 3–19. Springer.
- Wen, M.; Kuba, J. G.; Lin, R.; Zhang, W.; Wen, Y.; Wang, J.; and Yang, Y. 2022. Multi-Agent Reinforcement Learning is a Sequence Modeling Problem. In *Advances in Neural Information Processing Systems*, volume 35, 16509–16521. Curran Associates, Inc.
- Yu, C.; Velu, A.; Vinitzky, E.; Gao, J.; Wang, Y.; Bayen, A. M.; and Wu, Y. 2022. The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games. In *Advances in Neural Information Processing Systems*, volume 35, 24611–24624. Curran Associates, Inc.